

Continuous Control I: Deterministic Policy Gradients & DDPG

Naeemullah Khan

naeemullah.khan@kaust.edu.sa



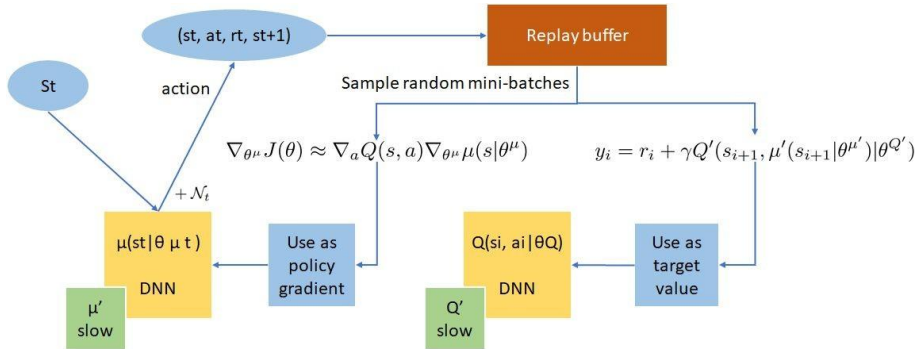
جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

May 16, 2026

Continuous Control I

Deterministic Policy Gradients & DDPG



1. Recap
2. Learning Outcomes
3. Motivation
4. Background
5. Method
6. Experiments
7. Extensions & Applications
8. Summary
9. References

Continuous Control: **Recap (Day 4)**

- ▶ **Actor-Critic** pairs a policy (actor) with a learned value function (critic) that supplies low-variance gradient signal
- ▶ The on-policy policy gradient:

$$\nabla_{\theta} \mathcal{J}(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) A^{\pi}(s_t, a_t) \right]$$

- ▶ A2C/A3C $\rightarrow$ TRPO $\rightarrow$ **PPO**: the workhorse of stable on-policy RL
- ▶ **Key limitation:** on-policy methods *discard all experience after each update* — extremely data-hungry in complex environments
- ▶ Stochastic policies also need a tractable $\log \pi_{\theta}$ — awkward for high-dimensional *continuous* actuators

- ▶ **Today we go off-policy and continuous.**
- ▶ **Note:** PPO already handles continuous actions *on-policy*; “Continuous Control I/II” is specifically the *off-policy, deterministic-policy* branch (DDPG/TD3 today) and the *max-entropy* branch (SAC, Day 6).
- ▶ **DPG** [4]: a policy-gradient update rule for *deterministic* policies $\mu_\theta(s)$ over continuous action spaces
- ▶ **DDPG** [5]: DPG made deep, by importing the DQN [6] tricks (replay buffer, target networks) — off-policy, sample-reusing
- ▶ The critic becomes an action-value $Q_\phi(s, a)$; the policy gradient flows *through the critic*: $\nabla_\theta Q_\phi(s, \mu_\theta(s))$
- ▶ We close with **TD3**, the direct fix for DDPG’s overestimation and brittleness — and the bridge to Day 6 (SAC)

- ▶ Explain why $\arg \max_a Q(s, a)$ is intractable in continuous action spaces, motivating a learned deterministic actor
- ▶ Derive the deterministic policy gradient from the stochastic policy gradient and identify the $\nabla_a Q$ term
- ▶ Describe how DDPG combines DPG with the DQN tricks (replay buffer, target networks) into an off-policy actor-critic
- ▶ Explain the role of soft target updates, batch normalisation, and added exploration noise (OU vs. Gaussian)
- ▶ State DDPG's failure modes (Q-overestimation, brittleness) and how TD3's three fixes address them

Alpha Go Zero [1]

selecting among possible moves for that piece. We represent the policy $\pi(a|s)$ by a $8 \times 8 \times 73$ stack of planes encoding a probability distribution over 4,672 possible moves. Each of the 8×8 positions identifies the square from which to “pick up” a piece. The first 56 planes encode. . .

OpenAI Five — Dota 2 [2]

floats and categorical values with hundreds of possibilities) each time step. We discretize the action space; on an average timestep our model chooses among 8,000 to 80,000 actions (depending on hero). For comparison Chess requires around one thousand. . .

Alpha Star [3]

observe and act next (Fig. 1a). This representation of actions in approximately 10^{26} possible choices at each step. Similar. . .

Extended Data Table 2 | Agent action space

Field	Description
Action type	Which action to execute. Some examples of actions are moving a unit, training a unit from a building, moving the camera, or no-op. See PySC2 for a full list ⁷
Selected units	Entities that will execute the action
Target	An entity or location in the map discretised to 256x256 targeted by the action
Queued	Whether to queue this action or execute it immediately
Repeat	Whether or not to issue this action multiple times
Delay	The number of game time-steps to wait until receiving the next observation

Control: choosing actions over time to drive a dynamical system toward desired behaviour — exactly the RL problem on an MDP $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$.

- ▶ **Discrete control:** the action set $\mathcal{A}$ is a *finite* list — move left/right, one of 4,672 Go moves, an Atari button. The policy picks one entry.
- ▶ **Continuous control:** the action is a *real-valued vector* $a \in \mathbb{R}^d$ (often bounded), e.g. $\mathcal{A} = [-1, 1]^d$.
 - Robot arm: joint torques; Locomotion: motor commands; Driving: steering angle + throttle.
 - The set is *uncountable* — there is no “list” to enumerate or take $\arg \max_a Q(s, a)$ over.

This lecture: how to do RL when actions are continuous.

- ▶ Go, Dota, and StarCraft have *enormous but still discrete* action sets — you can (in principle) enumerate and $\arg \max$ over them.
 - ▶ Robotics and physical control are *continuous*: joint torques, steering angles, velocities. The action set is uncountable — enumeration and $\arg \max_a Q(s, a)$ are no longer possible.
- ⇒ We need a method that learns a policy *directly* over a continuous action space — a first “deep” crack at this is **DDPG** [5].

DPG [4]

- Finds deterministic policy
- Applicable to continuous action space

DPG [4]

- Finds deterministic policy
- Applicable to continuous action space
- Not learning-based, can we do better?

DDPG (Deep DPG) in one sentence:

- ▶ Extends DPG (Deterministic Policy Gradients) [4] using deep learning,
- ▶ borrowing tricks from Deep Q-Learning (DQN) [6]

DDPG (Deep DPG) in one sentence:

- ▶ Extends DPG (Deterministic Policy Gradients) [4] using deep learning,
- ▶ borrowing tricks from Deep Q-Learning (DQN) [6]
- ▶ Contribution: model-free, off-policy, actor-critic approach that allows us to better learn deterministic policies on continuous action space

DDPG (Deep DPG) is a *model-free, off-policy, actor-critic* algorithm that combines:

- ▶ **DPG** (Deterministic Policy Gradients) [4]: works over continuous action domain, not learning-based
- ▶ **DQN** (Deep Q-Learning) [6]: learning-based, doesn't work over continuous action domain

In Q-learning, we find deterministic policy by

$$\mu^{k+1}(s) = \boxed{\arg \max_a} Q^{\mu^k}(s, a)$$

In Q-learning, we find deterministic policy by

$$\mu^{k+1}(s) = \boxed{\arg \max_a} Q^{\mu^k}(s, a)$$

Problem: In large discrete action space or continuous action space, we can't plug in every possible action to find the optimal action!

In Q-learning, we find deterministic policy by

$$\mu^{k+1}(s) = \boxed{\arg \max_a} Q^{\mu^k}(s, a)$$

Problem: In large discrete action space or continuous action space, we can't plug in every possible action to find the optimal action!

Solution: Learn a function approximator for $\arg \max$, via gradient descent

$$\mu^{k+1}(s) = \pi_{\boxed{\theta}}(s)$$

► **Goal:**

Derive a gradient update rule to learn deterministic policy

► **Goal:**

Derive a gradient update rule to learn deterministic policy

► **Idea:**

Adapt the stochastic policy gradient formulation for deterministic policies

- ▶ Vanilla Stochastic Policy Gradient:

Source: [13]

- Vanilla Stochastic Policy Gradient:

$$\theta^* = \arg \max_{\theta} \underbrace{\mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\sum_t r(\mathbf{s}_t, \mathbf{a}_t) \right]}_{J(\theta)}$$

Source: [13]

► Vanilla Stochastic Policy Gradient:

$$\theta^* = \arg \max_{\theta} \underbrace{\mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\sum_t r(\mathbf{s}_t, \mathbf{a}_t) \right]}_{J(\theta)}$$

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}(\tau)} \left[\left(\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t) \right) \left(\sum_{t=1}^T r(\mathbf{s}_t, \mathbf{a}_t) \right) \right]$$

Source: [13]

► Vanilla Stochastic Policy Gradient:

$$\theta^* = \arg \max_{\theta} \underbrace{\mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\sum_t r(\mathbf{s}_t, \mathbf{a}_t) \right]}_{J(\theta)}$$

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}(\tau)} \left[\left(\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t) \right) \left(\sum_{t=1}^T r(\mathbf{s}_t, \mathbf{a}_t) \right) \right]$$
$$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$$

Source: [13]

- Vanilla Stochastic Policy Gradient:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}(\tau)} \left[\left(\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t \mid \mathbf{s}_t) \right) \left(\sum_{t=1}^T r(\mathbf{s}_t, \mathbf{a}_t) \right) \right]$$

model-free

Not trivial to compute!

Source: [13]

- Vanilla Stochastic Policy Gradient with Monte-Carlo Sampling:

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_{i,t} \mid \mathbf{s}_{i,t}) \right) \left(\sum_{t=1}^T r(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}) \right)$$

Source: [13]

- Vanilla Stochastic Policy Gradient with Monte-Carlo Sampling:

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_{i,t} \mid \mathbf{s}_{i,t}) \right) \left(\sum_{t=1}^T r(\mathbf{s}_{i,t}, \mathbf{a}_{i,t}) \right)$$

Problem: Point Estimate – High Variance!

Source: [13]

► Vanilla Stochastic Policy Gradient:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}(\tau)} \left[\left(\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t, \mathbf{s}_t) \right) \left(\sum_{t=1}^T r(\mathbf{s}_t, \mathbf{a}_t) \right) \right]$$

Source: [13]

► Vanilla Stochastic Policy Gradient:

$$\begin{aligned}\nabla_{\theta} J(\theta) &= \mathbb{E}_{\tau \sim \pi_{\theta}(\tau)} \left[\left(\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t, \mathbf{s}_t) \right) \left(\sum_{t=1}^T r(\mathbf{s}_t, \mathbf{a}_t) \right) \right] \\ &= \mathbb{E}_{s \sim \rho^{\pi}, a \sim \pi_{\theta}} \left[\nabla_{\theta} \log_{\pi_{\theta}}(a | s) \boxed{Q^{\pi_{\theta}}(s, a)} \right]\end{aligned}$$

True value function is still not trivial to compute

Source: [13]

► Vanilla Stochastic Policy Gradient:

$$\begin{aligned}\nabla_{\theta} J(\theta) &= \mathbb{E}_{\tau \sim \pi_{\theta}(\tau)} \left[\left(\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t, \mathbf{s}_t) \right) \left(\sum_{t=1}^T r(\mathbf{s}_t, \mathbf{a}_t) \right) \right] \\ &= \mathbb{E}_{s \sim \rho^{\pi}, a \sim \pi_{\theta}} \left[\nabla_{\theta} \log \pi_{\theta}(a | s) \boxed{Q^{\pi_{\theta}}(s, a)} \right]\end{aligned}$$

True value function is still not trivial to compute, but we can approximate it with a parameterized function:

$$\nabla_{\theta} J(\pi_{\theta}) = \mathbb{E}_{s \sim \rho^{\pi}, a \sim \pi_{\theta}} \left[\nabla_{\theta} \log \pi_{\theta}(a | s) \boxed{Q^w(s, a)} \right]$$

Source: [13]

► Stochastic Policy Gradient (Actor-Critic)

$$\nabla_{\theta} J(\pi_{\theta}) = \mathbb{E}_{s \sim \rho^{\pi}, a \sim \pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a | s) Q^w(s, a)]$$

► Stochastic Policy Gradient (Actor-Critic)

$$\nabla_{\theta} J(\pi_{\theta}) = \mathbb{E}_{s \sim \rho^{\pi}, a \sim \pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a | s) Q^w(s, a)]$$

Actor: Policy function π_{θ}

► Stochastic Policy Gradient (Actor-Critic)

$$\nabla_{\theta} J(\pi_{\theta}) = \mathbb{E}_{s \sim \rho^{\pi}, a \sim \pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a | s) Q^w(s, a)]$$

Actor: Policy function π_{θ}

Critic: Value function Q^w , which provides guidance to improve the actor

- ▶ Deterministic Policy Gradient (Actor-Critic)

► Deterministic Policy Gradient (Actor-Critic)

Objective:

$$J(\pi_{\theta}) = \int_{\mathcal{S}} \rho^{\pi}(s) r(s, \pi_{\theta}(s)) ds = \mathbb{E}_{s \sim \rho^{\pi}} [r(s, \pi_{\theta}(s))]$$

► Deterministic Policy Gradient (Actor-Critic)

Objective:

$$J(\pi_\theta) = \int_{\mathcal{S}} \rho^\pi(s) r(s, \pi_\theta(s)) ds = \mathbb{E}_{s \sim \rho^\pi} [r(s, \pi_\theta(s))]$$

Policy Gradient:

$$\nabla_\theta J(\pi_\theta) = \int_{\mathcal{S}} \rho^\pi(s) \nabla_\theta \pi_\theta(s) \nabla_a Q^w(s, a)|_{a=\pi_\theta(s)} ds = \mathbb{E}_{s \sim \rho^\pi} [\nabla_\theta \pi_\theta(s) \nabla_a Q^w(s, a)|_{a=\pi_\theta(s)}]$$

Deterministic Policy Gradient (Actor-Critic)

Objective:

$$J(\pi_\theta) = \int_{\mathcal{S}} \rho^\pi(s) r(s, \pi_\theta(s)) ds = \mathbb{E}_{s \sim \rho^\pi} \left[r(s, \pi_\theta(s)) \right]$$

Policy Gradient:

$$\nabla_\theta J(\pi_\theta) = \int_{\mathcal{S}} \rho^\pi(s) \nabla_\theta \pi_\theta(s) \nabla_a Q^w(s, a) \Big|_{a=\pi_\theta(s)} ds = \mathbb{E}_{s \sim \rho^\pi} \left[\nabla_\theta \pi_\theta(s) \nabla_a Q^w(s, a) \Big|_{a=\pi_\theta(s)} \right]$$

Stochastic Policy Gradient:

$$\nabla_{\theta} J(\pi_{\theta}) = \mathbb{E}_{s \sim \rho^{\pi}, a \sim \pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a | s) Q^w(s, a)]$$

Deterministic Policy Gradient:

$$\nabla_{\theta} J(\pi_{\theta}) = \int_{\mathcal{S}} \rho^{\pi}(s) \nabla_{\theta} \pi_{\theta}(s) \nabla_a Q^w(s, a) \Big|_{a=\pi_{\theta}(s)} ds = \mathbb{E}_{s \sim \rho^{\pi}} \left[\nabla_{\theta} \pi_{\theta}(s) \nabla_a Q^w(s, a) \Big|_{a=\pi_{\theta}(s)} \right]$$

Stochastic Policy Gradient:

$$\nabla_{\theta} J(\pi_{\theta}) = \mathbb{E}_{s \sim \rho^{\pi}, a \sim \pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a | s) Q^w(s, a)]$$

Deterministic Policy Gradient:

functions!

DDPG: Use deep learning to learn both

$$\nabla_{\theta} J(\pi_{\theta}) = \mathbb{E}_{s \sim \rho^{\pi}} \left[\nabla_{\theta} \pi_{\theta}(s) \nabla_a Q^w(s, a) \Big|_{a=\pi_{\theta}(s)} \right]$$

- ▶ **DPG:** Formulates an update rule for deterministic policies, so that we can learn deterministic policy on continuous action domain

- ▶ **DPG:** Formulates an update rule for deterministic policies, so that we can learn deterministic policy on continuous action domain
- ▶ **DQN:** Enables learning value functions with neural nets, with two tricks:
 - Target Network
 - Replay Buffer

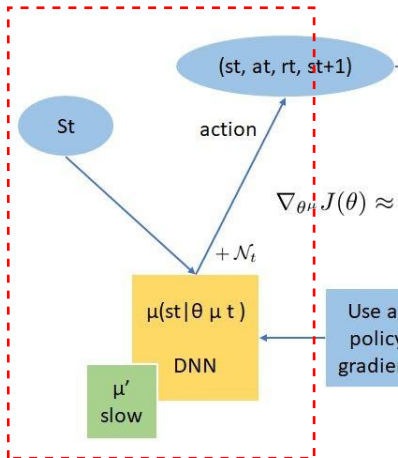
- ▶ **DPG:** Formulates an update rule for deterministic policies, so that we can learn deterministic policy on continuous action domain
Model-Free, Actor-Critic
- ▶ **DQN:** Enables learning value functions with neural nets, with two tricks:
 - Target Network
 - Replay Buffer – **Off-Policy**

- ▶ **DPG:** Formulates an update rule for deterministic policies, so that we can learn deterministic policy on continuous action domain
Model-Free, Actor-Critic
- ▶ **DQN:** Enables learning value functions with neural nets, with two tricks:
 - Target Network
 - Replay Buffer – **Off-Policy**
- ▶ **DDPG:** Learn both the policy and the value function in DPG with neural networks, with DQN tricks!

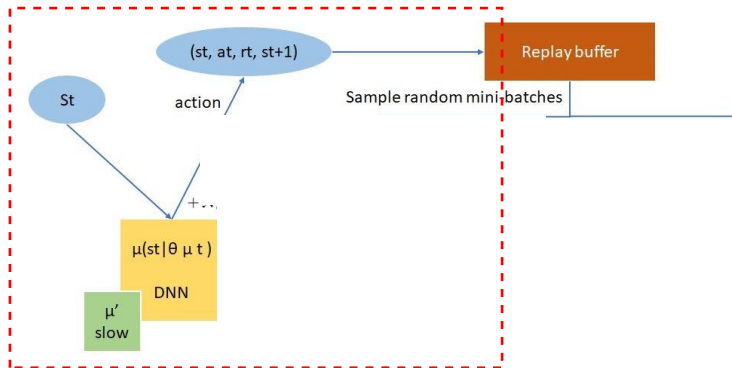
$\mu(s \mid \theta^\mu)$ Policy (Actor) Network
Deterministic, Continuous Action Space

$\mu(s \mid \theta^\mu)$	Policy (Actor) Network Deterministic, Continuous Action Space
$Q(s, a \mid \theta^Q)$	Value (Critic) Network

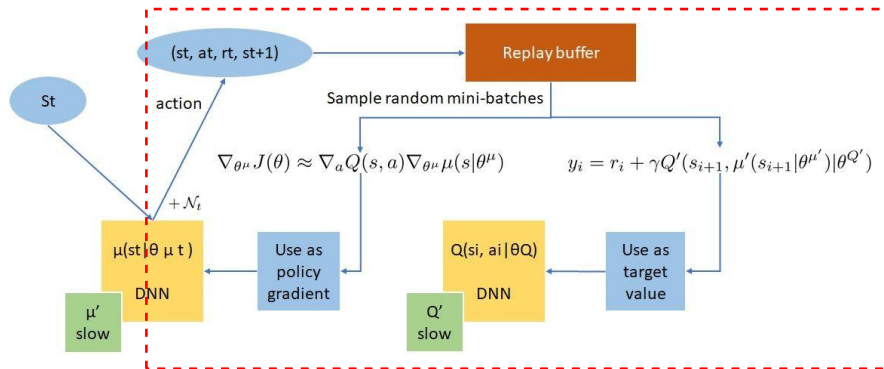
$\mu(s \mid \theta^\mu)$	Policy (Actor) Network Deterministic, Continuous Action Space
$Q(s, a \mid \theta^Q)$	Value (Critic) Network
$\mu'(s \mid \theta^{\mu'}), Q'(s, a \mid \theta^{Q'})$	Target Policy and Value Networks



Adapted from [14]



Adapted from [14]



Adapted from [14]

Algorithm 1 DDPG algorithm

Randomly initialize critic network $Q(s, a|\theta^Q)$ and actor $\mu(s|\theta^\mu)$ with weights θ^Q and θ^μ .

Initialize target network Q' and μ' with weights $\theta^{Q'} \leftarrow \theta^Q, \theta^{\mu'} \leftarrow \theta^\mu$

Initialize replay buffer R

for episode = 1, M **do**

 Initialize a random process $\mathcal{N}$ for action exploration

 Receive initial observation state s_1

for t = 1, T **do**

 Select action $a_t = \mu(s_t|\theta^\mu) + \mathcal{N}_t$ according to the current policy and exploration noise

 Execute action a_t and observe reward r_t and observe new state s_{t+1}

 Store transition (s_t, a_t, r_t, s_{t+1}) in R

 Sample a random minibatch of N transitions (s_i, a_i, r_i, s_{i+1}) from R

 Set $y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'}))|\theta^{Q'}$

 Update critic by minimizing the loss: $L = \frac{1}{N} \sum_i (y_i - Q(s_i, a_i|\theta^Q))^2$

 Update the actor policy using the sampled policy gradient:

$$\nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_i \nabla_a Q(s, a|\theta^Q)|_{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu} \mu(s|\theta^\mu)|_{s_i}$$

 Update the target networks:

$$\theta^{Q'} \leftarrow \tau \theta^Q + (1 - \tau) \theta^{Q'}$$

$$\theta^{\mu'} \leftarrow \tau \theta^\mu + (1 - \tau) \theta^{\mu'}$$

end for

end for

Replay buffer

“Soft” target
network
update

Algorithm 1 DDPG algorithm

Randomly initialize critic network $Q(s, a|\theta^Q)$ and actor $\mu(s|\theta^\mu)$ with weights θ^Q and θ^μ .

Initialize target network Q' and μ' with weights $\theta^{Q'} \leftarrow \theta^Q$, $\theta^{\mu'} \leftarrow \theta^\mu$

Initialize replay buffer R

for episode = 1, M do

Initialize a random process $\mathcal{N}$ for action exploration

Receive initial observation state s_1

for $t = 1, T$ do

Select action $a_t = \mu(s_t|\theta^\mu) + \mathcal{N}_t$ according to the current policy and exploration noise

Execute action a_t and observe reward r_t and observe new state s_{t+1}

Store transition (s_t, a_t, r_t, s_{t+1}) in R

Sample a random minibatch of N transitions (s_i, a_i, r_i, s_{i+1}) from R

Set $y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'})|\theta^{Q'})$

Update critic by minimizing the loss: $L = \frac{1}{N} \sum_i (y_i - Q(s_i, a_i|\theta^Q))^2$

Update the actor policy using the sampled policy gradient:

$$\nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_i \nabla_a Q(s, a|\theta^Q)|_{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu} \mu(s|\theta^\mu)|_{s_i}$$

Update the target networks:

$$\theta^{Q'} \leftarrow \tau \theta^Q + (1 - \tau) \theta^{Q'}$$

$$\theta^{\mu'} \leftarrow \tau \theta^\mu + (1 - \tau) \theta^{\mu'}$$

end for

end for

DQN copies the online weights into the target network *periodically* (a hard update every C steps). DDPG instead lets the targets *slowly track* the online networks every step:

$$\theta^{Q'} \leftarrow \tau \theta^Q + (1 - \tau) \theta^{Q'} \qquad \theta^{\mu'} \leftarrow \tau \theta^{\mu} + (1 - \tau) \theta^{\mu'}$$

- ▶ $\tau \ll 1$ (typically $\tau = 10^{-3}$): targets change very slowly.
- ▶ Constrains the TD target to move slowly $\Rightarrow$ greatly improves stability of learning, at the cost of slower value propagation.

Add noise for
exploration

Algorithm 1 DDPG algorithm

Randomly initialize critic network $Q(s, a|\theta^Q)$ and actor $\mu(s|\theta^\mu)$ with weights θ^Q and θ^μ .
Initialize target network Q' and μ' with weights $\theta^{Q'} \leftarrow \theta^Q$, $\theta^{\mu'} \leftarrow \theta^\mu$
Initialize replay buffer R

for episode = 1, M **do**

Initialize a random process $\mathcal{N}$ for action exploration

Receive initial observation state s_1

for $t = 1, T$ **do**

Select action $a_t = \mu(s_t|\theta^\mu) + \mathcal{N}_t$ according to the current policy and exploration noise

Execute action a_t and observe reward r_t and observe new state s_{t+1}

Store transition (s_t, a_t, r_t, s_{t+1}) in R

Sample a random minibatch of N transitions (s_i, a_i, r_i, s_{i+1}) from R

Set $y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'}))|\theta^{Q'}$

Update critic by minimizing the loss: $L = \frac{1}{N} \sum_i (y_i - Q(s_i, a_i|\theta^Q))^2$

Update the actor policy using the sampled policy gradient:

$$\nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_i \nabla_a Q(s, a|\theta^Q)|_{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu} \mu(s|\theta^\mu)|_{s_i}$$

Update the target networks:

$$\theta^{Q'} \leftarrow \tau \theta^Q + (1 - \tau) \theta^{Q'}$$

$$\theta^{\mu'} \leftarrow \tau \theta^\mu + (1 - \tau) \theta^{\mu'}$$

end for

end for

A deterministic actor $\mu(s \mid \theta^\mu)$ has *no* exploration of its own. DDPG explores by adding noise to the behaviour policy:

$$a_t = \mu(s_t \mid \theta^\mu) + \mathcal{N}_t$$

- ▶ **DDPG** [5]: $\mathcal{N}_t$ from an *Ornstein–Uhlenbeck process* [12] — temporally *correlated* noise, well suited to physical control with inertia/momentum.
- ▶ **Later finding (TD3 [7])**: uncorrelated zero-mean *Gaussian* noise works just as well and is simpler — now the common default.

Value Network Update

Algorithm 1 DDPG algorithm

Randomly initialize critic network $Q(s, a|\theta^Q)$ and actor $\mu(s|\theta^\mu)$ with weights θ^Q and θ^μ .
 Initialize target network Q' and μ' with weights $\theta^{Q'} \leftarrow \theta^Q$, $\theta^{\mu'} \leftarrow \theta^\mu$
 Initialize replay buffer R
for episode = 1, M **do**
 Initialize a random process $\mathcal{N}$ for action exploration
 Receive initial observation state s_1
 for t = 1, T **do**
 Select action $a_t = \mu(s_t|\theta^\mu) + \mathcal{N}_t$ according to the current policy and exploration noise
 Execute action a_t and observe reward r_t and observe new state s_{t+1}
 Store transition (s_t, a_t, r_t, s_{t+1}) in R
 Sample a random minibatch of N transitions (s_i, a_i, r_i, s_{i+1}) from R
 Set $y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'}))|_{\theta^{Q'}}$
 Update critic by minimizing the loss: $L = \frac{1}{N} \sum_i (y_i - Q(s_i, a_i|\theta^Q))^2$
 Update the actor policy using the sampled policy gradient:

$$\nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_i \nabla_a Q(s, a|\theta^Q)|_{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu} \mu(s|\theta^\mu)|_{s_i}$$

Update the target networks:

$$\theta^{Q'} \leftarrow \tau \theta^Q + (1 - \tau) \theta^{Q'}$$

$$\theta^{\mu'} \leftarrow \tau \theta^\mu + (1 - \tau) \theta^{\mu'}$$

end for
end for

Policy Network Update

Algorithm 1 DDPG algorithm

Randomly initialize critic network $Q(s, a|\theta^Q)$ and actor $\mu(s|\theta^\mu)$ with weights θ^Q and θ^μ .

Initialize target network Q' and μ' with weights $\theta^{Q'} \leftarrow \theta^Q$, $\theta^{\mu'} \leftarrow \theta^\mu$

Initialize replay buffer R

for episode = 1, M **do**

 Initialize a random process $\mathcal{N}$ for action exploration

 Receive initial observation state s_1

for t = 1, T **do**

 Select action $a_t = \mu(s_t|\theta^\mu) + \mathcal{N}_t$ according to the current policy and exploration noise

 Execute action a_t and observe reward r_t and observe new state s_{t+1}

 Store transition (s_t, a_t, r_t, s_{t+1}) in R

 Sample a random minibatch of N transitions (s_i, a_i, r_i, s_{i+1}) from R

 Set $y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'})|\theta^{Q'})$

 Update critic by minimizing the loss: $L = \frac{1}{N} \sum_i (y_i - Q(s_i, a_i|\theta^Q))^2$

 Update the actor policy using the sampled policy gradient:

$$\nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_i \nabla_a Q(s, a|\theta^Q)|_{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu} \mu(s|\theta^\mu)|_{s_i}$$

Update the target networks:

$$\theta^{Q'} \leftarrow \tau \theta^Q + (1 - \tau) \theta^{Q'}$$

$$\theta^{\mu'} \leftarrow \tau \theta^\mu + (1 - \tau) \theta^{\mu'}$$

end for

end for

Algorithm 1 Deep Q-learning with Experience Replay

Initialize replay memory $\mathcal{D}$ to capacity N
 Initialize action-value function Q with random weights
for episode = 1, M **do**
 Initialize sequence $s_1 = \{x_1\}$ and preprocessed sequenced $\phi_1 = \phi(s_1)$
 for $t = 1, T$ **do**
 With probability ϵ select a random action a_t
 otherwise select $a_t = \max_a Q^*(\phi(s_t), a; \theta)$
 Execute action a_t in emulator and observe reward r_t and image x_{t+1}
 Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$
 Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in $\mathcal{D}$
 Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from $\mathcal{D}$
 Set $y_j = \begin{cases} r_j & \text{for terminal } \phi_{j+1} \\ r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta) & \text{for non-terminal } \phi_{j+1} \end{cases}$
 Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ according to equation 3
 end for
end for

Algorithm 1 Deep Q-learning with Experience Replay

Initialize replay memory $\mathcal{D}$ to capacity N
 Initialize action-value function Q with random weights

for episode = 1, M **do**

 Initialize sequence $s_1 = \{x_1\}$ and preprocessed sequenced $\phi_1 = \phi(s_1)$

for $t = 1, T$ **do**

DDPG: Policy Network, learned with Deterministic Policy Gradient

 Execute action a_t in emulator and observe reward r_t and image x_{t+1}

 Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$

 Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in $\mathcal{D}$

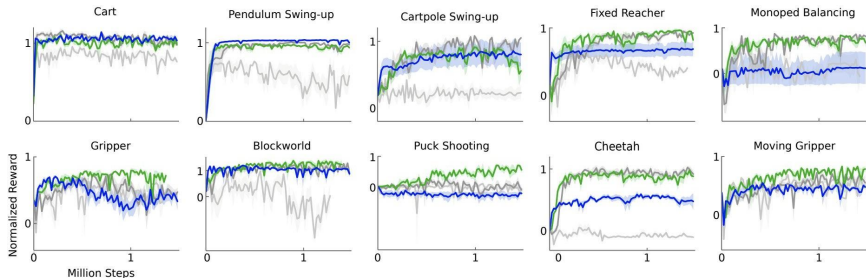
 Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from $\mathcal{D}$

 Set $y_j = \begin{cases} r_j & \text{for terminal } \phi_{j+1} \\ r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta) & \text{for non-terminal } \phi_{j+1} \end{cases}$

 Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ according to equation 3

end for

end for



Light Grey: Original DPG

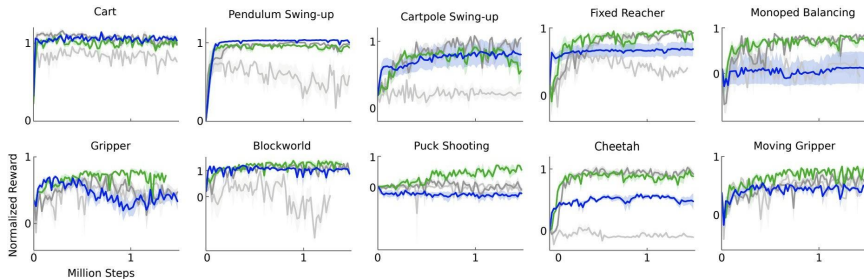
Dark Grey: Target Network

Green: Target Network + Batch Norm

Blue: Target Network from pixel-only

Figure: DDPG [5]

Do target networks and batch norm matter?



Light Grey: Original DPG

Dark Grey: Target Network

Green: Target Network + Batch Norm

Blue: Target Network from pixel-only

Figure: DDPG [5]

Experiments — Is DDPG better than DPG?

Normalised return (1.0
≈ planning baseline)
across 30+ tasks.

DDPG columns:
low-dim (*lowd*) and
pixel (*pix*) inputs.

DPG columns: original
control (*cntrl*).

environment	$R_{av,lowd}$	$R_{best,lowd}$	$R_{av,pix}$	$R_{best,pix}$	$R_{av,cntrl}$	$R_{best,cntrl}$
blockworld1	1.156	1.511	0.466	1.299	-0.080	1.260
blockworld3da	0.340	0.705	0.889	2.225	-0.139	0.658
canada	0.303	1.735	0.176	0.688	0.125	1.157
canada2d	0.400	0.978	-0.285	0.119	-0.045	0.701
cart	0.938	1.336	1.096	1.258	0.343	1.216
cartpole	0.844	1.115	0.482	1.138	0.244	0.755
cartpoleBalance	0.951	1.000	0.335	0.996	-0.468	0.528
cartpoleParallelDouble	0.549	0.900	0.188	0.323	0.197	0.572
cartpoleSerialDouble	0.272	0.719	0.195	0.642	0.143	0.701
cartpoleSerialTriple	0.736	0.946	0.412	0.427	0.583	0.942
cheetah	0.903	1.206	0.457	0.792	-0.008	0.425
fixedReacher	0.849	1.021	0.693	0.981	0.259	0.927
fixedReacherDouble	0.924	0.996	0.872	0.943	0.290	0.995
fixedReacherSingle	0.954	1.000	0.827	0.995	0.620	0.999
gripper	0.655	0.972	0.406	0.790	0.461	0.816
gripperRandom	0.618	0.937	0.082	0.791	0.557	0.808
hardCheetah	1.311	1.990	1.204	1.431	-0.031	1.411
hopper	0.676	0.936	0.112	0.924	0.078	0.917
hyq	0.416	0.722	0.234	0.672	0.198	0.618
movingGripper	0.474	0.936	0.480	0.644	0.416	0.805
pendulum	0.946	1.021	0.663	1.055	0.099	0.951
reacher	0.720	0.987	0.194	0.878	0.231	0.953
reacher3daFixedTarget	0.585	0.943	0.453	0.922	0.204	0.631
reacher3daRandomTarget	0.467	0.739	0.374	0.735	-0.046	0.158
reacherSingle	0.981	1.102	1.000	1.083	1.010	1.083
walker2d	0.705	1.573	0.944	1.476	0.393	1.397
torcs	-393.385	1840.036	-401.911	1876.284	-911.034	1961.600

Table: DDPG [5]

DDPG often reaches a strong *best* run, but its *average* return is far lower — a large gap signals instability across seeds/episodes:

Environment	$R_{av, lowd}$	$R_{best, lowd}$
cart	0.938	1.336
pendulum	0.946	1.021
canada	0.303	1.735
gripper	0.618	0.972
torcs	-393.4	1840.0

- ▶ Simple tasks (cart, pendulum): average $\approx$ best — stable.
- ▶ Harder tasks (canada, gripper) and especially **torcs**: average collapses while best stays high $\Rightarrow$ **high variance / brittleness**.
- ▶ This overestimation-driven instability is exactly what **TD3** later targets.

How well does Q estimate the true returns?

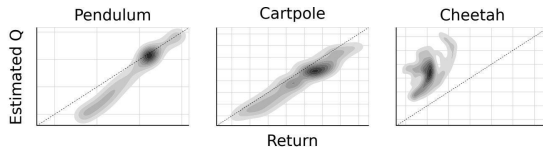


Figure 3: Density plot showing estimated Q values versus observed returns sampled from test episodes on 5 replicas. In simple domains such as pendulum and cartpole the Q values are quite accurate. In more complex tasks, the Q estimates are less accurate, but can still be used to learn competent policies. Dotted line indicates unity, units are arbitrary. (Figure: DDPG [5].)

The experiments showed DDPG *overestimates* Q and is brittle. **TD3** (Twin Delayed DDPG) keeps the deterministic off-policy actor-critic but adds three fixes:

1. **Clipped double-Q (twin critics):** learn two critics, use the *minimum* for the TD target to curb overestimation bias:

$$y = r + \gamma \min_{i=1,2} Q_{\theta'_i}(s', \mu_{\theta'}(s')) + \epsilon$$

2. **Target policy smoothing:** $\epsilon \sim \text{clip}(\mathcal{N}(0, \sigma), -c, c)$ — regularizes the target so the actor cannot exploit sharp Q peaks.
3. **Delayed policy updates:** update the actor (and targets) less often than the critics (e.g. every 2 critic steps) for lower-variance policy updates.

Takeaway: TD3 is the deterministic successor to DDPG; its clipped double-Q idea carries directly into SAC (next lecture).

- ▶ **Make the actor an explicit $\arg \max$ of a convex critic** — avoids a separate policy network entirely:
 - *Continuous Deep Q-Learning with Model-based Acceleration (NAF)* [9]
 - *Input Convex Neural Networks* [10]
- ▶ **Stochastic policy search** — replace the deterministic actor with an entropy-regularised stochastic one (the Day 6 path):
 - *Soft Actor-Critic* [8]

often used to perform the maximization in Q-learning. A commonly used algorithm in such settings, deep deterministic policy gradient (DDPG) (Lillicrap et al., 2015), provides for sample-efficient learning but is notoriously challenging to use due to its extreme brittleness and hyperparameter sensitivity (Duan et al., 2016; Henderson et al., 2017).

ily to very complex, high-dimensional tasks, such as the Humanoid benchmark (Duan et al., 2016) with 21 action dimensions, where off-policy methods such as DDPG typically struggle to obtain good results (Gu et al., 2016). SAC also avoids the complexity and potential instability associ-



Learning to Drive in a Day [11]

We selected a simple continuous action domain model-free reinforcement learning algorithm: deep deterministic policy gradients (DDPG) [5], to show that an off-the-shelf reinforcement learning algorithm with no task-specific adaptation is capable of solving the MDP posed in Section III-A.

DDPG consists of two function approximators: a critic $Q : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$, which estimates the value $Q(s, a)$ of the expected cumulative discounted reward upon using action a in state s , trained to satisfy the Bellman equation

$$Q(s_t, a_t) = r_{t+1} + \gamma(1 - d_t)Q(s_{t+1}, \pi(s_{t+1})),$$

under a policy given by the actor $\pi : \mathcal{S} \rightarrow \mathcal{A}$, which attempts to estimate a Q -optimal policy $\pi(s) = \arg \max_a Q(s, a)$; here $(s_t, a_t, r_{t+1}, d_{t+1}, s_{t+1})$ is an experience tuple, a transition

Continuous Control: **Summary**

DQN + DPG → **DDPG** → +clipped double-Q / delayed updates → **TD3**
→ +max-entropy → **SAC** (Day 6)

Algorithm	On-policy?	Action space	Policy	Stability
DQN	No (replay)	Discrete	Implicit ($\arg \max Q$)	Medium
DPG	No	Continuous	Deterministic	— (not deep)
DDPG	No (replay)	Continuous	Deterministic	Brittle
TD3	No (replay)	Continuous	Deterministic	Improved

- ▶ **DDPG = DPG + DQN.** Bypass the intractable $\arg \max_a Q$ by training a second network (the actor) to output the maximising action; the gradient flows through the critic
- ▶ Tricks that made DDPG work:
 - **Replay buffer & target networks** (from DQN) — off-policy, stable TD targets
 - **Soft target updates** $\theta' \leftarrow \tau\theta + (1 - \tau)\theta'$ instead of DQN's hard periodic copy
 - **Batch normalisation** — transfers across tasks with different state scales
 - **Added action noise** (OU / Gaussian) — exploration for a deterministic policy
- ▶ **TD3** fixes DDPG's Q-overestimation with clipped double-Q, target-policy smoothing, and delayed policy updates

- ▶ **Overestimation bias:** the single critic, maximised by the actor, systematically over-estimates Q (the experiments showed $R_{av} \ll R_{best}$)
- ▶ **Hyperparameter brittleness:** sensitive to learning rates, τ , noise scale, and network architecture; results vary widely across random seeds
- ▶ **Deterministic exploration:** relies entirely on injected noise — weak, undirected exploration in sparse-reward tasks
- ▶ **Scaling:** struggles on high-dimensional, complex tasks compared with later methods (TD3, SAC)
- ▶ **Two-network instability:** actor and critic form a coupled feedback loop that can diverge if either lags

- ▶ DDPG and TD3 give us *off-policy, continuous control* — but with a **deterministic** policy and only weak, injected exploration

- ▶ DDPG and TD3 give us *off-policy, continuous control* — but with a **deterministic** policy and only weak, injected exploration
- ▶ **Day 6 — Soft Actor-Critic (SAC):** maximum-entropy RL
 - Stochastic policy with an explicit **entropy bonus** in the objective:
$$\mathcal{J}(\pi) = \mathbb{E}[\sum_t r_t + \alpha \mathcal{H}(\pi(\cdot|s_t))]$$
 - Entropy drives *directed* exploration — no hand-tuned OU/Gaussian noise schedule
 - Reuses what we built today: replay buffer, target networks, and **TD3's clipped double-Q** critic
 - The deterministic policy gradient $\nabla_{\theta} Q(s, \mu_{\theta}(s))$ generalises to the **reparameterised** stochastic gradient
- ▶ Takeaway: SAC is the entropy-regularised, stochastic sibling of TD3 — everything from today carries over

Continuous Control: **References**

- [1] Silver, D., Schrittwieser, J., Simonyan, K., et al. (2017).
Mastering the Game of Go without Human Knowledge (AlphaGo Zero).
Nature, 550, 354–359.
- [2] OpenAI: Berner, C., Brockman, G., et al. (2019).
Dota 2 with Large Scale Deep Reinforcement Learning (OpenAI Five).
arXiv:1912.06680.
- [3] Vinyals, O., Babuschkin, I., Czarnecki, W. M., et al. (2019).
Grandmaster Level in StarCraft II using Multi-Agent Reinforcement Learning (AlphaStar).
Nature, 575, 350–354.
- [4] Silver, D., Lever, G., Heess, N., Degris, T., Wierstra, D., & Riedmiller, M. (2014).
Deterministic Policy Gradient Algorithms.
ICML 2014, PMLR 32, 387–395.
- [5] Lillicrap, T. P., Hunt, J. J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D., & Wierstra, D. (2015).
Continuous Control with Deep Reinforcement Learning.
arXiv:1509.02971; *ICLR 2016*.

- [6] Mnih, V., Kavukcuoglu, K., Silver, D., et al. (2015).
Human-Level Control through Deep Reinforcement Learning.
Nature, 518(7540), 529–533.
- [7] Fujimoto, S., van Hoof, H., & Meger, D. (2018).
Addressing Function Approximation Error in Actor-Critic Methods (TD3).
ICML 2018, PMLR 80, 1587–1596; arXiv:1802.09477.
- [8] Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018).
Soft Actor-Critic: Off-Policy Maximum Entropy Deep RL with a Stochastic Actor.
ICML 2018, PMLR 80, 1856–1865; arXiv:1801.01290.
- [9] Gu, S., Lillicrap, T., Sutskever, I., & Levine, S. (2016).
Continuous Deep Q-Learning with Model-Based Acceleration (NAF).
ICML 2016, PMLR 48, 2829–2838; arXiv:1603.00748.
- [10] Amos, B., Xu, L., & Kolter, J. Z. (2017).
Input Convex Neural Networks.
ICML 2017, PMLR 70, 146–155; arXiv:1609.07152.

- [11] Kendall, A., Hawke, J., Janz, D., et al. (2018).
Learning to Drive in a Day.
arXiv:1807.00412; *ICRA* 2019.
- [12] Uhlenbeck, G. E., & Ornstein, L. S. (1930).
On the Theory of the Brownian Motion.
Physical Review, 36(5), 823–841.
- [13] Levine, S. CS285: Deep Reinforcement Learning, UC Berkeley.
<https://rail.eecs.berkeley.edu/deeprlcourse/>
- [14] Garg, A. CSC2621: Reinforcement Learning in Robotics, University of Toronto.
<https://animesh.garg.tech/CSC2621-W20/>

These slides have been adapted from

- ▶ Animesh Garg, [CSC2621: Reinforcement Learning in Robotics](#), University of Toronto [14]
- ▶ Sergey Levine, [CS285: Deep Reinforcement Learning](#), UC Berkeley [13] — the DPG derivation